

Willians Cassiano de Freitas Abreu

Redes LSTM com custo CTC aplicadas à decodificação de digitação por gestos

Brasil

2018, v-1.0

Willians Cassiano de Freitas Abreu

Redes LSTM com custo CTC aplicadas à decodificação de digitação por gestos

Um estudo sobre digitação por gestos, *Long Short Term Memory Networks*, *Connectio-nist Temporal Classification* e modelos de síntese gestual baseados na minimização do *jerk*

Universidade Federal do Amazonas – UFAM

Instituto de Computação

Bacharelado em Engenharia de Software

Orientador: Marco Antônio Pinheiro de Cristo

Coorientador: Juan Gabriel Colonna

Brasil

2018, v-1.0

Willians Cassiano de Freitas Abreu
Redes LSTM com custo CTC aplicadas à decodificação de digitação por gestos/
Willians Cassiano de Freitas Abreu. – Brasil, 2018, v-1.0-
53 p. : il. (algumas color.) ; 30 cm.

Orientador: Marco Antônio Pinheiro de Cristo

Monografia (Bacharelado) – Universidade Federal do Amazonas – UFAM
Instituto de Computação
Bacharelado em Engenharia de Software, 2018, v-1.0.

1. digitação. 2. gestos. 3. LSTM. 4. CTC. 4. síntese. I. Marco Antônio Pinheiro de Cristo. II. Universidade Federal do Amazonas. III. Instituto de Computação. IV. Bacharelado em Engenharia de Software

CDU 02:141:005.7

Willians Cassiano de Freitas Abreu

Redes LSTM com custo CTC aplicadas à decodificação de digitação por gestos

Um estudo sobre digitação por gestos, *Long Short Term Memory Networks*, *Connectio-nist Temporal Classification* e modelos de síntese gestual baseados na minimização do *jerk*

Trabalho aprovado. Brasil, 21 de junho de 2018:

Marco Antônio Pinheiro de Cristo
Orientador

Eulanda Miranda dos Santos
Convidada

Brasil
2018, v-1.0

Agradecimentos

Este trabalho é o resultado de alguns anos de curiosidade, trabalho, sorte – e principalmente, oportunidades. Oportunidades geradas por pessoas que encontrei ao longo do caminho, que acreditaram em mim, e sem as quais essa jornada incrível não seria possível.

Agradeço ao professor Marco Cristo, por ter me aceitado como orientando, pela presteza e disponibilidade, demonstradas nas incontáveis horas dispendidas esclarecendo dúvidas em sua sala e remotamente, que culminaram na elaboração deste trabalho. Ao professor Juan Colonna, coorientador, colega de trabalho e amigo, pelas colaborações e idéias que resultaram em diversas contribuições desta monografia, além das dicas assertivas de redação científica e matemática.

Ao professor Moisés Carvalho, pelas conversas nos primeiros semestres, que despertaram em mim a vontade de ser um cientista na computação. Ao professor Raimundo Barreto, por ter deixado a porta de sua sala sempre aberta para um calouro ávido por conhecimento. Ao professor Reginaldo Carvalho, pela oportunidade de aprender e pelos conselhos pessoais e profissionais perfeitamente dosados.

Aos meus colegas de trabalho e de turma. E a todos aqueles com os quais cruzei caminhos nestes anos de universidade, e que de uma forma ou de outra me ajudaram técnica, científica e emocionalmente a enfrentar as dificuldades durante todo esse tempo. Obrigado a vocês pela confiança, companherismo, apoio e carinho.

“There is a theory which states that if ever anyone discovers exactly what the Universe is for and why it is here, it will instantly disappear and be replaced by something even more bizarre and inexplicable. There is another theory which states that this has already happened.

– Douglas Adams

Resumo

O problema da digitação por gestos foi abordado pela primeira vez na década de 1980, como uma alternativa de entrada de dados mais eficiente que a digitação tecla a tecla. Desde então, com a crescente popularidade dos dispositivos móveis, esse tipo de interação tornou-se cada vez mais comum. Em comparação a métodos tradicionais, esta forma de entrada é mais ruidosa, no sentido em que um mesmo sinal de entrada pode ser interpretado de diferentes formas. Assim, novos métodos para este tipo de entrada têm sido propostos com intuito de melhorar a capacidade de reconhecimento sob as mais diversas condições de uso e características dos usuários. Entre estes métodos, um de particular sucesso, devido a sua conhecida robustez a ruídos, são as redes neurais recorrentes. Neste trabalho, implementamos um modelo de reconhecimento de palavras digitadas com gestos, baseado em um método estado-da-arte, que emprega uma rede RNN com memória, ou seja, com células LSTM (Long- Short-Term Memory). Além disso, para prover dados de treino para o modelo, também implementamos um modelo de síntese gestual capaz de produzir os gestos correspondentes às palavras que um usuário iria digitar da forma mais similar possível àquela que seria captada a partir de uma entrada real produzida por um ser humano. Treinamos e avaliamos o modelo implementado em um léxico com cerca de 62 mil palavras, usando uma função de custos que avalia a sequência completa de caracteres reconhecidos da palavra (CTC — Connectionist Temporal Classification). Por fim, decodificamos as probabilidades resultantes usando um transdutor de estado finito (FST — Finite State Transducer) construído a partir de uma Trie, obtida a partir do léxico, por meio de um algoritmo de busca aproximada, o Beam Search. O modelo implementado apresentou um desempenho compatível com o reportado na literatura, com cerca de 97% de reconhecimento correto para a tarefa avaliada.

Palavras-chaves: digitação por gestos. LSTM. connectionist temporal classification. síntese gestual.

Abstract

The keyboard gesture decoding problem was first approached in the 1980s as a more efficient data entry alternative than conventional typing. Since then, with the increasing popularity of mobile devices, this means of interaction has become increasingly common. Compared to traditional methods, this form of input is noisier in the sense that the same input signal can be interpreted in different ways. Thus, new methods for this type of input have been proposed in order to improve the recognition capacity under the most diverse conditions of use and characteristics of users. Among these methods, one particularly successful, due to its known robustness to noise, are recurrent neural networks. In this work, we implemented a gesture-based word recognition model, based on a state-of-the-art method, which uses an RNN with memory, that is, with Long-Short-Term Memory (LSTM) cells. In addition, to provide training data for the model, we also implemented a gestural synthesis model capable of producing the gestures corresponding to the words that a user would type in the most similar way possible to the one that would be captured from a real input produced by a human being. We train and evaluate the model implemented in a lexicon with approximately 62 thousand words, using a cost function that evaluates the complete sequence of recognized characters of the word (CTC - Connectionist Temporal Classification). Finally, we decode the resulting probabilities using a finite state transducer (FST) constructed from a Trie, obtained from the lexicon, by means of an approximate search algorithm, Beam Search. The implemented model achieved a performance compatible with that reported in the literature, with about 97% correct recognition for the evaluated task.

Key-words: keyboard gesture decoding. LSTM. connectionist temporal classification. gesture synthesis.

Lista de ilustrações

Figura 1 – Modelo do processo de digitação por gestos fim-a-fim	23
Figura 2 – Exemplo da palavra FORCE escrita utilizando Unistrokes	26
Figura 3 – Teclado <i>touchscreen</i> moderno	26
Figura 4 – Método Quickwriting digitando a letra <i>f</i>	27
Figura 5 – Cirrin utilizado para digitar a palavra <i>finished</i>	27
Figura 6 – Arranjo ATOMIK (à esquerda) e digitação com <i>stylus</i> (à direita) . . .	28
Figura 7 – Teclado ativado por <i>wipe</i> de Montgomery	28
Figura 8 – Diagrama de uma rede recorrente	30
Figura 9 – Diagrama de uma célula LSTM	31
Figura 10 – Exemplo de Trie	33
Figura 11 – Diagrama do modelo Bidirecional	36
Figura 12 – Diagrama do modelo de decoding fim-a-fim	36
Figura 13 – Em sentido horário: Modelagem onehot (ALSHARIF et al., 2015), $\delta =$ 0.01 e $\delta = 0.025$ para o mesmo ponto g_t	38
Figura 14 – Representação gráfica da matriz K para a palavra MARCO	39
Figura 15 – Exemplo de realizações de O para $s_1 = \{5, 10, 20\}$ da esquerda para a direita, com $K_i = m(\mathbf{M})$	39
Figura 16 – Exemplo gráfico de uma matriz Z , resultado da intercalação entre O (em azul) e V , (em preto). A linha tracejada indica o protótipo da trajetória entre os pontos de O	40
Figura 17 – Exemplo de trajetória G	41
Figura 18 – Histograma do comprimento das palavras no vocabulário	44
Figura 19 – Exemplo de trajetórias para a palavra ATTIC. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso a direita, a palavra iden- tificada foi SATIN.	45
Figura 20 – Exemplo de trajetórias para a palavra DUMBNESS. No exemplo à es- querda, o modelo decodifica corretamente. Já no caso à direita, a pala- vra identificada foi DIMNESS.	46
Figura 21 – Exemplo de trajetórias para a palavra ERRONEOUS. No exemplo à es- querda, o modelo decodifica corretamente. Já no caso à direita, a pala- vra identificada foi WRONGS.	46
Figura 22 – Exemplo de trajetórias para a palavra MISSTEPS. No exemplo à es- querda, o modelo decodifica corretamente. Já no caso à direita, a pala- vra identificada foi MISTED.	46

Figura 23 – Exemplo de trajetórias para a palavra SUGGESTIBLE . No exemplo à esquerda, o modelo decodifica corretamente. Já no caso à direita, a palavra identificada foi SUFFERING	46
Figura 24 – Exemplo de trajetórias para a palavras em português corretamente classificadas. Em sentido horário, começando acima à esquerda: PROPOSTA , ORDEM , PAPEL e NORTE	47

Lista de tabelas

Tabela 1 – Resultados obtidos para o <i>dataset</i> de teste na rodada final	45
--	----

Lista de abreviaturas e siglas

IHC	Interação Humano-Computador
SHARK	<i>SHorthand Aided Rapid Keyboarding</i>
CTC	<i>Connectionist Temporal Classification</i>
LSTM	<i>Long Short Term Memory</i>
FST	<i>Finite State Transducer</i>
GPU	<i>Graphics Processing Unit</i>
Seq2Seq	<i>Sequence-to-Sequence</i>
NMT	<i>Neural Machine Translation</i>

Sumário

1	INTRODUÇÃO	21
1.1	Definição do problema	22
1.2	Objetivos	23
1.3	Justificativa	23
1.4	Contribuições Principais	24
2	FUNDAMENTAÇÃO TEÓRICA E REVISÃO DA LITERATURA	25
2.1	Digitação por gestos	25
2.2	Algoritmos de reconhecimento	29
2.3	Redes LSTM	30
2.4	<i>Connectionist Temporal Classification</i>	32
2.5	Transdutores de estado finito e <i>Beam Search</i>	33
2.6	Modelos de síntese gestual	34
3	SÍNTESE E DECODIFICAÇÃO DE GESTOS	35
3.1	Rede LSTM com custo CTC	35
3.2	Extrator de características	37
3.3	Síntese	38
4	EXPERIMENTOS E RESULTADOS	43
4.1	Metodologia Experimental	43
4.1.1	Vocabulário	43
4.1.2	Treinamento	43
4.1.3	Avaliação	44
4.2	Reconhecimento de Palavras	45
4.3	Os Efeitos do Sintetizador sobre os Erros de Reconhecimento	45
4.4	Reconhecimento de Palavras em Português com Transferência de Aprendizado	47
5	CONCLUSÃO E TRABALHOS FUTUROS	49
	REFERÊNCIAS	51

1 Introdução

A escrita ainda é, apesar dos avanços em tecnologias multimídia, a principal forma de comunicação digital entre as pessoas, sendo responsável por boa parte do tráfego na rede mundial de computadores. Até aproximadamente quatro quintos da história da internet este conteúdo se originava, em sua maior parte, de teclados físicos conectados a computadores de mesa.

Com o aprimoramento das tecnologias móveis e sua adoção pelo grande público, novos métodos de interação puderam ser desenvolvidos, aproveitando-se da flexibilidade dessas novas plataformas. Como exemplo, podemos mencionar o uso pelas aplicações de geolocalização, acelerômetros, giroscópios, sensores de batimentos cardíacos, tecnologias de presença e – de mais interesse para este trabalho, as telas *touchscreen*.

As telas sensíveis ao toque introduziram uma gama de novas interações. Uma das aplicações destes novos métodos de interação é a digitação por gestos, que visa aumentar a performance do usuário ao permitir a inserção de uma ou mais palavras com apenas um gesto de primeira ordem na superfície sensível do dispositivo. Comparado a métodos tradicionais de entrada, como a digitação tecla-a-tecla, espera-se que o usuário consiga fornecer uma entrada com menor esforço físico e maior velocidade.

Entretanto, embora o teclado de gestos seja mais eficiente que sua contraparte física em um contexto *mobile*, a decodificação de gestos em palavras é consideravelmente mais complexa que a simples checagem de colisão em um teclado *touchscreen* comum.

Isto ocorre porque, além dos erros inerentes a um processo de entrada normal (erros gramaticais, erros de cognição, confusão fonética, proximidade das teclas, etc), outros erros específicos são observados como o uso de gestos similares para palavras diferentes (“Brasil” e “nescio” em um teclado QWERTY), a dificuldade em identificar que teclas correspondem às letras que o usuário quer digitar entre todas as presentes no caminho descrito pelo gesto e o emprego de diferentes estilos gestuais por diferentes usuários.

Devido à sua robustez a ruído, neste trabalho utilizamos redes neurais para realizar o mapeamento de gestos em palavras. Redes neurais do tipo recorrente foram escolhidas pela sua inerente capacidade de processar sequências com dependência temporal. Aplicamos o tipo de rede neural LSTM para evitar o problema de explosão-desaparecimento do gradiente, e usamos a busca aproximada *Beam Search* para eliminar palavras fora do vocabulário. A busca é realizada num FST no formato de uma Trie gerada a partir do léxico, com as probabilidades de transição atribuídas pela saída da LSTM.

Vamos introduzir a apresentar formalmente o problema neste capítulo. Em seguida,

apresentamos uma revisão da literatura relevante no Capítulo 2. No Capítulo 3 propomos um método para solucionar o problema baseado em (ALSHARIF et al., 2015) utilizando redes neurais recorrentes do tipo LSTM, *Beam Search* e FST-Trie. Uma descrição da metodologia utilizada e dos resultados obtidos podem ser encontradas no Capítulo 4. Por fim, discutimos a solução implementada e possíveis melhorias no Capítulo 5.

1.1 Definição do problema

Na Figura 1, temos um diagrama que apresenta os principais componentes da interação gestual fim-a-fim. Em linhas gerais, o usuário entra com um gesto que corresponde à palavra que ele deseja. Este gesto é codificado internamente para uma representação que precisa ser decodificada em uma palavra. Entre o processo de codificação e decodificação, podemos imaginar a inserção de ruído, que modela os muitos tipos de erros que podem ocorrer no processo como, por exemplo, a possibilidade do gesto não incluir todas as letras que ocorrem na palavra desejada. No modelo que vamos implementar, esse ruído deve ser inserido artificialmente para representar o ruído que poderia ser observado na interação real. Na figura, a informação decodificada é então entregue ao destino do processo de entrada. A seguir, apresentamos uma definição mais formal do problema.

Seja Σ o conjunto dos caracteres permitidos nas palavras de um vocabulário L , onde cada palavra $w = \{c_1, c_2, c_3, \dots, c_n : c_i \in \Sigma\}$ é uma sucessão ordenada de caracteres. Definimos o processo de codificação E , como uma função que mapeia todo w em L , para um gesto G . Representamos o gesto G como uma matriz $\mathbb{R}^{2 \times T}$ onde $G_{1,t}$ é a coordenada x do cursor no tempo t e, de maneira análoga $G_{2,t}$ é a coordenada y no tempo t , onde $t \in \mathbb{N}$ e T é a duração do gesto¹. Denotamos por g_t o vetor de posição do cursor no tempo t , ou de maneira análoga a coluna t em G .

Desta maneira um decodificador para o problema da digitação gestual é um estimador D para função $\hat{w} = \arg \max_w P(w \mid G)$, a função inversa de E , onde $\hat{w}$ é a palavra mais provável, G o gesto inserido pelo usuário e w a palavra que o usuário pretendia digitar.

Neste trabalho, estudamos os gestos produzidos quando o processo de codificação E resulta em uma trajetória G observada que é proveniente da intenção do usuário de conectar as teclas correspondentes aos caracteres da palavra w , na ordem em que estes ocorrem. O gesto equivalentemente a palavra MARCO, por exemplo, é gerado a partir de um toque inicial na tecla correspondente ao caracter M, movendo-se o dedo pressionado até a tecla A, daí à tecla R, para a tecla C, por fim elevando-se o dedo na tecla O. Neste trabalho, usaremos fonte monoespaçada para indicar caracteres e/ou símbolos de um alfabeto.

¹ Um modelo para E é discutido em detalhes na Seção 3.3

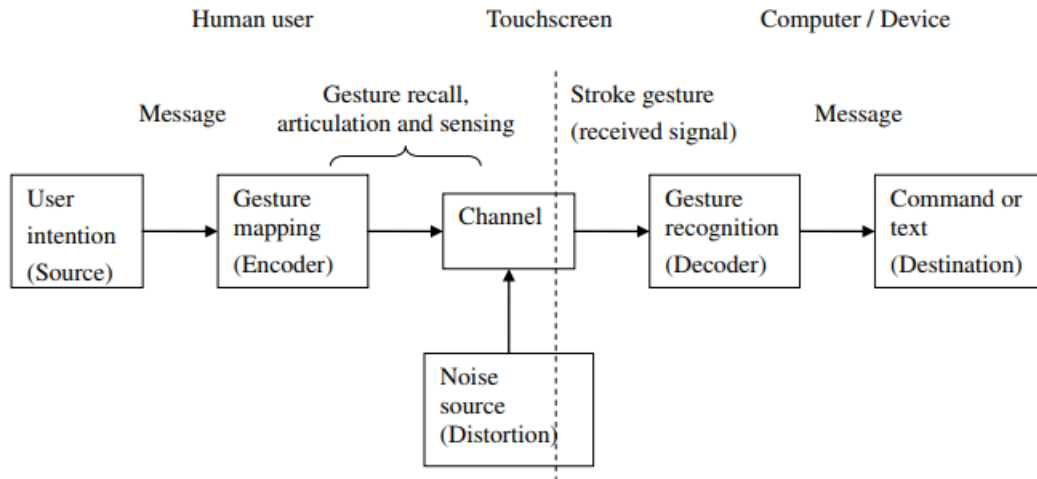


Figura 1 – Modelo do processo de digitação por gestos fim-a-fim

Em vários casos, é útil dar como entrada para o estimador D uma transformação de G , $e : G \rightarrow \mathbb{R}^{d \times T}$, onde d é o número de características utilizadas e T a duração do gesto. Desta forma, e é um extrator de características para G . Neste trabalho, implementamos um extrator de características probabilístico inspirado em QUINN; ZHAI. Este extrator se encontra descrito em detalhes na Seção 3.2.

1.2 Objetivos

Este trabalho tem como objetivo geral melhorar o reconhecimento de gestos usando um modelo neural, baseado em uma proposta feita em ALSHARIF et al.. Como objetivos específicos, temos a implementação de um modelo neural de decodificação de gestos e de um modelo de síntese de gestos.

1.3 Justificativa

O estimador D necessário para decodificação dos gestos para digitação precisa ser extremamente robusto a ruído, pois o processo de encoding E é probabilístico, não-determinístico e suscetível à introdução de perturbação em várias etapas, como erros cognitivos, erros do sistema motor, postura e características físicas do usuário (QUINN; ZHAI, 2018).

Nos últimos anos, boa parte das técnicas de reconhecimento de padrões e sequências vêm sendo substituídas por técnicas neurais, como redes totalmente conectadas, redes convolucionais fim-a-fim (WANG et al., 2012) e redes LSTM fim-a-fim (GRAVES; JAILTY, 2014). Uma das razões para essa transição é elevada tolerância destes modelo a

ruído. Como modelos neurais são aproximadores universais de função (HORNİK, 1991), tornam-se fortes candidatas a estimadores para D .

Nos casos onde tais funções possuem forte dependência temporal, i.e a observação no tempo t é fortemente acoplada a observação no tempo $t + 1$ do ponto de vista da classificação do padrão, redes recorrentes normalmente são aplicadas, por possuírem uma conexão temporal residual, permitindo ao modelo “lembrar” de entradas anteriores (GOODFELLOW et al., 2016).

Entretanto, devido ao problema de explosão-desaparecimento do gradiente decorrente da retropropagação temporal em cascata, redes recorrentes tradicionais não são muito utilizadas (PASCANU; MIKOLOV; BENGIO, 2012). Em seu lugar, utilizamos as redes neurais do tipo *Long Short Term Memory* que utilizam *gates* contínuos para apagar ou escrever no estado interno da célula, controlando a explosão e/ou desaparecimento do gradiente (HOCHREITER; SCHMIDHUBER, 1997). Portanto, baseando-nos no trabalho de ALSHARIF et al., aplicamos redes LSTM para o problema de decodificação gestual, utilizando uma função de custo CTC (*Connectionist Temporal Classification*) para lidar com sequências não segmentadas (GRAVES et al., 2006).

1.4 Contribuições Principais

Como resultado deste trabalho, além do processo de aprendizado de proposição e desenvolvimento de um projeto, em um contexto de pesquisa e finalização de curso, fornecemos as seguintes contribuições:

- Uma implementação código aberto de um modelo de síntese de gestos para treinamento de modelos baseada em QUINN; ZHAI, descrito na Seção 3.3.
- Uma implementação código aberto de um modelo decodificador de gestos utilizando redes recorrentes LSTM com função de custo CTC, busca em Trie com *Beam Search* baseada em ALSHARIF et al.; GRAVES; JAITLEY, descrito na Seção 3.1.
- Uma modelagem probabilística para um extrator de características e , derivada de QUINN; ZHAI, descrita na Seção 3.2.

2 Fundamentação teórica e revisão da literatura

Neste capítulo, apresentamos os conceitos básicos necessários para a compreensão deste trabalho, bem como descrevemos trabalhos relacionados. Em particular, abordamos tópicos relativos a digitação com gestos, redes recorrentes LSTM, função de custo CTC, Trie, Beam Search e modelos de síntese gestual.

2.1 Digitação por gestos

Apesar de estudos com interfaces centradas no toque existirem desde o início da década de 1990 ([CHATTY](#); [LECOANET, 1996](#); [PEDERSEN et al., 1995](#); [WOLF](#); [RHYNE, 1993](#)) e de algumas tentativas comerciais de introdução do touchscreen ao grande público como a start-up GO e o relógio Casio DB-1000, o sucesso dos smartphones Blackberry fizeram os teclados físicos tornarem-se o meio de inserção de texto dominante em telefones móveis neste período.

Nessa mesma época os dispositivos do tipo Palm, que adotavam telas sensíveis ao toque acionadas por canetas, apresentam sua tecnologia Graffiti ([HAWKINS](#); [SIPHER](#); [RON, 2002](#)), inspirados pelo trabalho de [GOLDBERG](#); [RICHARDSON](#) no qual o usuário utilizava um traço único para representar uma letra.

Estes primeiros métodos trabalhavam a nível de caracter, designando um único gesto para cada um dos símbolos disponíveis. No seu trabalho, [GOLDBERG](#); [RICHARDSON](#) constroem um vocabulário de gestos a partir de 5 tipos básicos de traço. Variando orientações e direção podia se reconhecer até 40 símbolos distintos. Um exemplo de digitação por gestos utilizando Unistrokes é apresentado na Figura 2.

Nestas abordagens, embora os protótipos de gestos fossem projetados para se assemelhar aos símbolos do alfabeto, nem sempre isso era possível, e sua natureza abstrata requeria que os usuários memorizassem as associações entre o gesto e o caracter, que poderiam não ser aparentes.

Melhorias futuras, ainda baseadas em caracter surgiram posteriormente, como o Jot e o Graffiti 2 ([SEARS](#); [ARORA, 2002](#)), possibilitando a inserção de múltiplos traços, resolvendo as ambiguidades introduzidas pelos símbolos f , j e x , por exemplo.

Apesar desses avanços, é apenas no final dos anos 2000 que o lançamento do iPhone pela Apple populariza a utilização de dispositivos móveis com interfaces *touchscreen* no dia-a-dia. Acredita-se que incapacidade do hardware existente anteriormente em prover



Figura 2 – Exemplo da palavra FORCE escrita utilizando Unistrokes

uma boa experiência do usuário possa ter sido a causa da adoção tardia dos dispositivos com interação baseada em toque.

O teclado de toque utilizado nos smartphones *touchscreen* modernos normalmente possui o arranjo QWERTY, e é modelado propositalmente para se assimilar ao objeto físico (SUTSKEVER et al., 2013), em forma e interação, diminuindo a curva de aprendizado. Na Figura 3, temos um exemplo de tal teclado. O usuário insere uma palavra tocando com os dedos ou um dispositivo apontador na célula da matriz de botões virtuais correspondente ao caracter que deseja inserir, um de cada vez.

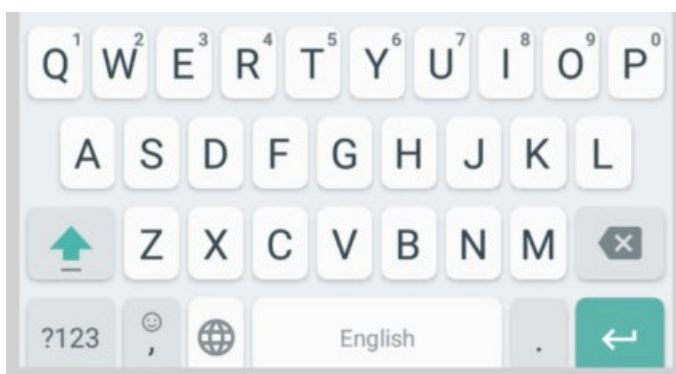


Figura 3 – Teclado *touchscreen* moderno

Devido à sua natureza mais flexível, os dispositivos sensíveis a toque permitem uma gama de métodos de interações muito maior do que os seus equivalentes físicos. Este trabalho explora os mecanismos de interação baseados em gestos, mais adequados para as restrições das aplicações móveis. Em especial, métodos para a entrada de texto baseada em padrões gestuais.

Nas interfaces de toque tradicionais o usuário normalmente se utiliza de traços de ordem zero, (segundo a taxonomia de ZHAI et al.) para digitar. Dado que o usuário precisa atingir um ponto de contato individual para cada caracter que deseja inserir, esses métodos são inerentemente lentos. Isso acontece pois estes métodos objetivam emular a interação em um teclado físico, de maneira a diminuir a curva de aprendizado.

Sistemas como Jot, Graffiti e o Unistroke, embora trouxessem avanços interessantes como a independência visual-espacial (ZHAI et al., 2012) e utilizarem gestos de primeira e alta ordem, ainda eram lentos, por serem baseados em caracter.

O método de escrita Quickwriting para interfaces *stylus* foi introduzido no final

da década de 90 e utilizava quadrantes com letras agrupadas para que o usuário os conectasse (PERLIN, 1998). Neste sistema os gestos começavam e terminavam na mesma zona de descanso no centro, e um algoritmo simples de checagem em tabelas era utilizado para realizar o reconhecimento baseado na posição (x, y) do cursor e quadrantes de controle (*shifting*). Quickwriting era baseado em gestos de primeira ordem, e permitia tanto a escrita de palavras quanto de caracteres. Todavia, devido ao seu *design*, ser fundamentalmente um método de inserção de caracteres.



Figura 4 – Método Quickwriting digitando a letra *f*

No mesmo ano, Mankoff e Abowd desenvolvem o primeiro teclado de gestos a nível de palavras chamado Cirrin (MANKOFF; ABOWD, 1998), utilizando um *layout* radial e dispondo as letras em um arranjo que objetivava minimizar a distância percorrida pela caneta na superfície. Seu trabalho era focado em métodos de interação para pessoas com necessidades especiais ou dificuldades motoras.

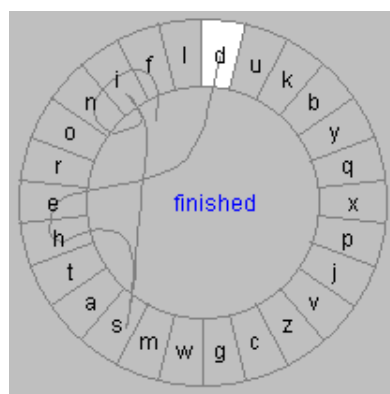


Figura 5 – Cirrin utilizado para digitar a palavra *finished*

Em 2003, Zhai e Kristensson introduzem um teclado gestual de palavras com arranjo horizontal, também utilizando gestos de primeira ordem (um único traço) (ZHAI; KRISTENSSON, 2003). Os autores, inspirados pela proximidade de algumas letras em palavras ou segmentos de palavras em língua inglesa no teclado, imaginaram uma técnica na qual o usuário poderia digitar conectando as células desejadas da matriz com um único

traço. Em seu trabalho, Zhai e Kristensson traçam um paralelo desta forma de escrita com a estenografia.

Este método, denominado SHARK, sigla para *Shorthand Aided Rapid Keyboarding*, adaptava um arranjo diferente do QWERTY denominado ATOMIK (ZHAI; HUNTER; SMITH, 2002), o que tinha o propósito de aproximar teclas que aparecem em bigramas frequentes em língua inglesa e minizar a distância média de um traço requerido para inserir uma palavra.

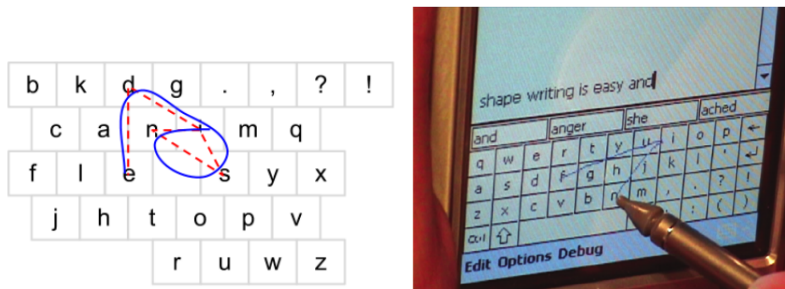


Figura 6 – Arranjo ATOMIK (à esquerda) e digitação com *stylus* (à direita)

Não obstante o método SHARK ser frequentemente tido como o primeiro dos teclados horizontais com digitação por gestos, o trabalho de Montgomery (MONTGOMERY, 1982) em 1982 pode ser considerado o seminal para digitação por gestos. Embora seu dispositivo fosse físico e a viabilidade de implementação do seu trabalho fosse duvidosa para a tecnologia da época, os autores em ZHAI; KRISTENSSON o reconhecem como um pesquisador à frente do seu tempo, pioneiro neste tipo de interação e lamentam seu impacto limitado nos 20 anos que se seguiram à publicação. O teclado *wipe*, como Montgomery chamou seu método, trabalhava com um arranjo frequentista como o ATOMIK.

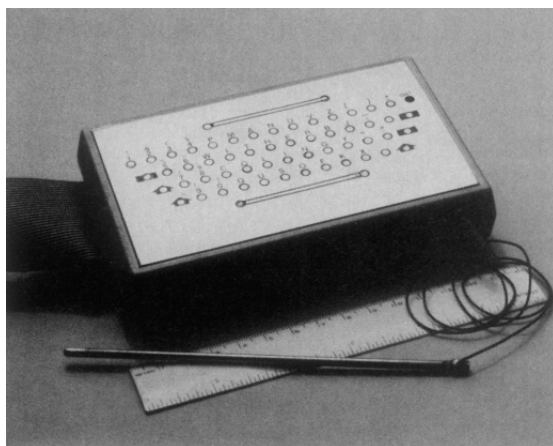


Figura 7 – Teclado ativado por *wipe* de Montgomery

Um dos problemas com o arranjo horizontal percebido pelos autores do SHARK era a ambiguidade, que era praticamente inexistente em *layouts* como o Quickwriting e o Cirrin. Apesar disso, o arranjo horizontal aproveitava melhor o espaço, eliminando a

necessidade de uma zona de descanso. No ano seguinte, Zhai e Kristensson apresentaram SHARK2 (KRISTENSSON; ZHAI, 2004), uma extensão do seu primeiro trabalho, com melhorias nos algoritmos de reconhecimento, expansão do vocabulário e otimizações em seu *design* original.

Nos anos seguintes, várias realizações comerciais desses estudos surgiram, já adotando o arranjo QWERTY e disponibilizando as tecnologias para o amplo público. Destacam-se os teclados implementados pela ShapeWriter, SlideIT, Swype, T9 Trace, FlexT9, and TouchPal, SwiftKey e Google. Estas empresas combinaram as técnicas de reconhecimento de padrão já utilizadas com modelos de linguagem ¹ para realizar previsões mais assertivas e assim diminuir a taxa de erros e aumentar a quantidade de palavras por minuto que um usuário médio era capaz de digitar. Assim, teclados de digitação por gestos se popularizaram e hoje são incorporados como método de entrada padrão no sistema operativo móvel mais amplamente utilizado, o Android.

2.2 Algoritmos de reconhecimento

O reconhecimento de gestos é um exemplo de pesquisa multidisciplinar. Existem diferentes ferramentas para reconhecimento de gestos, e os métodos vão desde modelagem estatística, visão computacional e reconhecimento de padrões até processamento de imagens e sistemas conexionistas (MITRA; ACHARYA, 2007). Em geral, soluções para o problema de reconhecimento de gestos são baseadas em modelagem estatística, como PCA, Modelos Ocultos de Markov, Filtros de Kalman, filtros de partículas e algoritmos de condensação. Até mesmo máquinas de estado finito foram efetivamente empregadas na modelagem de gestos humanos (MITRA; ACHARYA, 2007).

Técnicas de visão computacional e reconhecimento de padrões envolvendo extração de características, detecção de objetos, clustering e classificação também foram usadas com sucesso em muitos sistemas de reconhecimento de gestos (MITRA; ACHARYA, 2007). Técnicas de processamento de imagem como análise e detecção de forma, textura, cor, movimento, fluxo óptico, aprimoramento de imagem, segmentação e modelagem de contorno também se mostraram eficazes. Abordagens conexionistas, envolvendo *multi-layer perceptron*, *time-delay neural networks* e redes de função de base radial, também foram utilizados no reconhecimento de gestos (MITRA; ACHARYA, 2007).

Nos primeiros trabalhos, como o de ZHAI; KRISTENSSON, os autores utilizam técnicas já conhecidas de reconhecimento de escrita cursiva para decodificar gestos. No teclado SHARK, um algoritmo de programação dinâmica chamado *elastic matching* foi

¹ Modelo descritor de uma linguagem que possibilita a inferência de informações de interesse sobre a linguagem como, por exemplo, a probabilidade de ocorrência de uma palavra, dado que uma ou mais palavras foram previamente observadas

implementado para realizar o pareamento entre os protótipos das palavras do vocabulário (caminhos ideais) para o gesto do usuário, calculando desta maneira $P(w | G)$.

Entre as técnicas empregadas em reconhecimento de escrita cursiva aplicadas à decodificação por gestos podemos mencionar técnicas de pré-processamento como *smoothing*, *filtering*, *stroke connection* e normalização; Técnicas de reconhecimento como árvores de decisão baseadas em características, sequências visu-temporais, pareamento de *templates*, *dynamic time warping* e análise por síntese (TAPPERT; SUEN; WAKAHARA, 1990).

Embora tais técnicas tenham atingido relativo sucesso, com o avanço da capacidade de computação dos dispositivos móveis, aliado ao desenvolvimento de técnicas cada vez mais sofisticadas de aprendizagem de máquina, métodos clássicos têm sido deixados de lado para dar lugar à abordagens conexionistas profundas, como as LSTMs, que abordaremos neste trabalho.

2.3 Redes LSTM

As redes recorrentes são um tipo específico de rede neural introduzida na década de 1980 para solucionar problemas com sequências. Devido à existência de uma conexão resíduo-temporal para um estado interno do qual a saída para cada tempo t é dependente, este tipo de arquitetura permite à rede neural utilizar uma entrada de um tempo $t-m$ para produzir sua saída no tempo t . A Figura 8 exibe uma rede recorrente desdobrada no tempo. A recorrência é desdobrada em uma sucessão de cópias da rede, em que a informação de saída no instante de tempo t é fornecida como entrada adicional para a próxima cópia no instante $t+1$. Ao fim, tanto a entrada quanto a saída da rede correspondem a séries temporais de entradas e saídas.

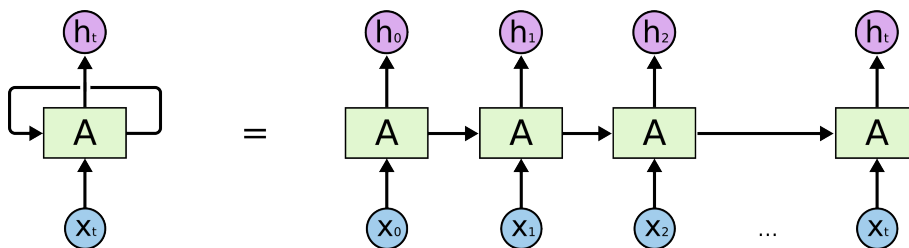


Figura 8 – Diagrama de uma rede recorrente

Métodos neurais estimam sua função alvo por meio de ajustes graduais em seus parâmetros. Esses ajustes são realizados utilizando uma variante do gradiente descendente, o algoritmo de retro-propagação (*backpropagation*). Este é um método de ajuste de parâmetros, introduzido por RUMELHART; HINTON; WILLIAMS. Em suma, este método atualiza os pesos da rede neural através de uma técnica de perturbação, representada pela Equação 2.1:

$$W \rightarrow W' = W - \eta \delta \frac{\partial \sigma(C)}{\partial W} X \quad (2.1)$$

onde W é a matriz de pesos de uma camada realacionada com as conexões entre as entradas X da camada e suas células constituintes, δ é o vetor de créditos (ou descrédito) relacionados com estas células que indicam o quanto elas são responsáveis pelo atual grau de erro do modelo, η a taxa de aprendizado e C a função de custo, ou função de erro, modificada por uma função de ativação σ . A taxa de aprendizado estabelece quão grande será o passo a ser dado pelo algoritmo durante as iterações realizadas para buscar o ponto de mínimo da superfície de erro descrita por $\sigma(C)$. Por sua vez, a função de ativação σ introduz uma transformação não linear em C . Devido ao seu desdobramento no tempo, redes recorrentes usualmente sofrem com o problema do desaparecimento/explosão do gradiente (PASCANU; MIKOLOV; BENGIO, 2012), o que torna seu treinamento extremamente complexo.

Isso acontece quando a matriz W é atualizada diversas vezes numa mesma propagação, tendendo seu valor para 0 ou infinito. Para solucionar esse problema, as redes LSTM introduzem novas matrizes de pesos, responsáveis por apagar, ler ou escrever o estado interno da célula, o que mantém o gradiente controlado durante a retropropagação.

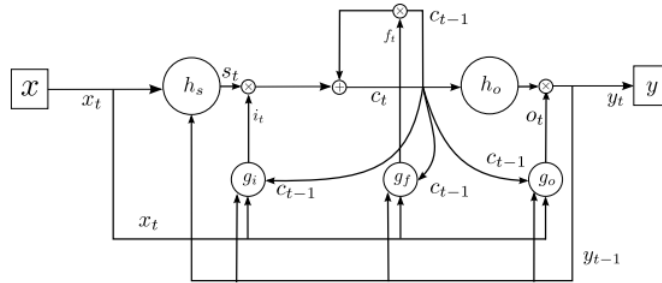


Figura 9 – Diagrama de uma célula LSTM

Formalmente, para cada entrada $x \in \mathbb{R}^{d \times T}$, a célula LSTM computa a seguinte função:

$$s_t = h_s(W_s(y_{t-1} + x_t)) \quad (2.2)$$

$$i_t = g_i(W_i(y_{t-1} + x_t + c_{t-1})) \quad (2.3)$$

$$f_t = g_f(W_f(y_{t-1} + x_t + c_{t-1})) \quad (2.4)$$

$$c_t = i_t \odot s_t + c_{t-1} \odot f_t \quad (2.5)$$

$$o_t = g_o(W_o(x_t + y_{t-1} + c_t)) \quad (2.6)$$

$$y_t = o_t \odot h_o(c_t) \quad (2.7)$$

onde W_s , W_i , W_f , W_o são as matrizes de pesos que controlam, respectivamente: o estado interno, o portão de entrada, o portão de esquecimento e o portão de saída. Temos h_i , g_i ,

g_f , g_o sendo as funções de ativação sobre, respectivamente: o estado, a entrada, o portão de esquecimento, e o portão de saída. E por fim, c_t , s_t , y_t como o estado final, o estado candidato, e a saída da rede no tempo t . A Figura 9 apresenta um diagrama demonstrando a relação entre todas estas variáveis. Durante o treinamento, o modelo consegue aprender como usar as portas para lembrar de informações do passado o que, na prática, o torna mais robusto ao problema de desaparecimento e explosão de gradientes, além de fornecer uma certa capacidade de memorização arbitrária dos sinais observados.

Em algumas situações pode ser útil processar a sequência nas duas “direções”, i.e tanto no sentido de produção dos dados $x = [x_1, x_2, x_3, \dots, x_T]$ quanto no sentido oposto $x = [x_T, x_{T-1}, x_{T-2}, \dots, x_1]$. Neste caso, utilizamos duas redes LSTMs, cada uma processando x em um sentido, e para cada tempo t , combinamos a saída das duas. Pode-se usar diversas funções de combinação entre as duas LSTMs, como a multiplicação elemento a elemento, concatenação, média elemento a elemento, entre outras. Estas RNNs são conhecidas como LSTMs bidirecionais. Em nosso trabalho combinamos a saída das duas redes utilizando a soma elemento a elemento.

2.4 Connectionist Temporal Classification

Existem diversas funções de custo utilizadas em redes neurais. Em geral a função de custo é dependente da tarefa que se quer realizar. Em problemas de classificação – por exemplo, é comum a utilização da entropia cruzada (SHORE; JOHNSON, 1980), em problemas de aproximação numérica – erro quadrático, além de diversas outras, como funções de perda ponderada.

Neste trabalho, baseando-nos em ALSHARIF et al., utilizamos a função de custo *Connectionist Temporal Classification* (GRAVES et al., 2006), uma função que foca em etiquetar sequências temporais não-segmentadas. A função CTC é escolhida pois em geral é muito trabalhoso alinhar, segmentar e etiquetar sequências.

Em nosso contexto, desejamos mapear uma sequência de entrada $G = [g_1, g_2, g_3, \dots, g_t, \dots, g_T]$ em uma sequência de saída $w = [c_1, c_2, c_3, \dots, c_j, \dots, c_m]$, onde G é o gesto do usuário, T o comprimento do gesto, w uma palavra composta de caracteres $c_j \in \Sigma$, e m o comprimento de w . Podemos observar três problemas aqui: tanto m quanto n podem variar; a razão entre m e T pode variar; não conhecemos um alinhamento (mapeamento) entre um dado g_t e seu c_j correspondente.

A função de custo CTC resolve esse problema introduzindo uma nova classe de saída para a rede neural – o símbolo branco λ , e computando as probabilidades para cada combinação entre os símbolos do alfabeto de saída $\Sigma \cup \{\lambda\}$. O cálculo de probabilidade para cada combinação possível de saída torna-se rapidamente computacionalmente intratável, mas pode ser resolvido satisfatoriamente com um algoritmo de programação dinâmica

(GRAVES et al., 2006).

Desta forma a probabilidade de uma palavra w , dado um gesto G , é dada por:

$$P(w | G) = \sum_{A \in A_w} \prod_{t=1}^T p_t(c | G), \quad (2.8)$$

onde A_w é o conjunto dos alinhamentos válidos (GRAVES et al., 2006) de w e $p_t(c | G)$ é probabilidade de um dado caracter ou o símbolo branco no tempo t , estimado por nossa rede recorrente. Como demonstrado por GRAVES et al., toda a função é derivável, de maneira que podemos treinar uma rede neural para minimizar $-\log(P(w | G))$ utilizando gradiente descendente.

2.5 Transdutores de estado finito e Beam Search

Após o treinamento, em tempo de inferência a LSTM produz uma matriz $y \in \mathbb{R}^{|\Sigma^+| \times T}$, onde Σ^+ é $\Sigma \cup \{\lambda\}$, e T o comprimento do gesto. O valor da matriz $y_{i,j}$ é a probabilidade do elemento de Σ^+ do índice i corresponder a entrada G no tempo j . Esta matriz indica a correlação entre G e Σ^+ .

Uma das maneiras de decodificar a saída da rede é tomar o $\arg \max_i y_{i,j}$ para todos os tempos j , concatenando os caracteres correspondentes c_j para formar uma palavra w . Entretanto nem sempre este método nos dá palavras pertencentes ao vocabulário L das palavras válidas.

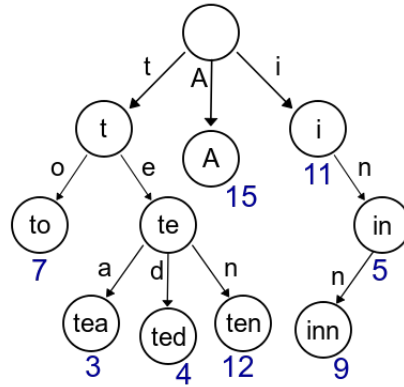


Figura 10 – Exemplo de Trie

Desta forma, para restringir a saída da rede, podemos tomar o nosso léxico e, a partir dele, construir um transdutor de estado finito (FST) em forma de *Trie*. Na Figura 10, vemos um exemplo de Trie para as palavras {TO, TEA, TED, TEN, A, I, IN, INN}. Como visto, na Trie, as arestas indicam caracteres das palavras. Os números em azul indicam custos do percurso até um nó associado com uma palavra observada no dicionário. Tais custos podem ser estimados de diferentes formas podendo representar, por exemplo,

o quão provável é aquela palavra. Por construção, a Trie só permite o processamento de prefixos de palavras vistas no dicionário, o que evita a possibilidade de estimativas desnecessárias.

No momento da decodificação, utilizamos a saída da rede para atribuir probabilidades às transições no FST. Ou seja, as arestas entre nós da Trie são ponderadas de acordo com a probabilidade de observação da sequência pela LSTM. Podemos então caminhar pela Trie, mantendo uma estimativa das k palavras mais prováveis no grafo, utilizando o algoritmo clássico de busca em largura, o *Beam Search*. Uma forma de implementar este algoritmo, no contexto de LSTMs, é descrita em detalhes por GRAVES; JAITLEY. Note que, neste caso, podemos dizer que a Trie torna a LSTM um modelo mais sofisticado de linguagem, ao incorporar informação de um léxico. Se ainda incorporarmos à estimativa de custo da Trie a probabilidade de ocorrência da palavra no dicionário, essa abordagem passa a integrar um modelo de linguagem de unigramas ao modelo de reconhecimento.

2.6 Modelos de síntese gestual

Para treinar uma rede neural profunda, em geral é necessária uma quantidade massiva de dados, que nem sempre estão facilmente acessíveis. Deste modo, sempre que possível, busca-se realizar aumento artificial dos dados (*data augmentation*). Quando trabalhamos com imagens é possível realizar recorte, certas rotações, adições de ruído, espelhamentos, entre outras modificações (KRIZHEVSKY; SUTSKEVER; HINTON, 2012). Já quando se trabalha com áudio, costuma-se modificar o *pitch*, volume e velocidade das amostras, por exemplo (GRAVES; JAITLEY, 2014).

Uma outra abordagem, é modelar o processo gerador dos dados (*data generation*), o que permite a produção de tantos exemplos quanto se queira para treinamento. Os algoritmos geradores utilizados devem ser aproximadores bons o suficiente dos fenômenos gerados (*end-effectors*). Usualmente eles não abrangem todo o processo gerador. Até porque, se possuíssemos um modelo gerador fiel e completo, um modelo reconhecedor seria facilmente derivável a partir do modelo sintetizador.

Em nosso problema, buscamos obter dados a partir de modelos de síntese gestual. Estes modelos procuram produzir amostras de gestos com a maior correlação possível com dados observados do mundo real, à partir de técnicas de otimização e controle (QUINN; ZHAI, 2018). Detalhes do algoritmo de síntese gestual utilizado neste trabalho, além de uma fundamentação matemática e teórica mais abrangente podem ser encontrados na Seção 3.3.

3 Síntese e Decodificação de Gestos

Neste capítulo, a partir da fundamentação teórica discutida no Capítulo 2, descrevemos o modelo decodificador utilizado neste trabalho para resolver o problema de digitação por gestos. Abordamos também nosso novo método de extração de características probabilístico, discutindo a arquitetura de entradas e saídas da rede. Finalmente, apresentamos o modelo de síntese de gestos que será usado para treinar nosso decodificador.

O modelo utilizado neste trabalho é uma reprodução do trabalho de decodificação de gestos para digitação proposto por [ALSHARIF et al.](#). Entretanto, há algumas diferenças de modelagem. Em suma:

- [ALSHARIF et al.](#) não utiliza um extrator de características (Seção 3.2) em tempo de treinamento/inferência, enquanto que nós utilizamos;
- [ALSHARIF et al.](#) utiliza uma camada escondida na LSTM de 400 neurônios, enquanto nós utilizamos 512;
- Por conseguinte, o modelo de [ALSHARIF et al.](#) possui 1.5 milhão de parâmetros, enquanto o nosso modelo possui 2.2 milhões.

Para além disso, em termos de métodos utilizados, mantêm-se as mesmas técnicas, como a aplicação da função de custo CTC, *BeamSearch* na Trie do vocabulário e o uso das probabilidades da saída da LSTM para atribuir probabilidades ao FST resultante.

3.1 Rede LSTM com custo CTC

Nesta seção descrevemos a nossa implementação de um estimador para $P(w | G)$, baseado no trabalho de [ALSHARIF et al.](#). Utilizamos uma rede neural do tipo LSTM com função de custo CTC, e treinamos em um dataset de gestos gerados artificialmente seguindo o algoritmo descrito na Seção 3.3.

A rede neural que utilizamos é uma LSTM bidirecional de apenas uma célula com matrizes W (estado interno) de dimensão **512**. A entrada para a rede é uma sequência de observações obtidas do extrator de características descrito na Seção 3.2 para cada tempo t de um gesto G . A figura 11 apresenta um diagrama que detalha graficamente o modelo.

A saída do estimador é, para cada tempo g_t , a probabilidade de um elemento de Σ^+ corresponder à decodificação correta de G no tempo t . Essas probabilidades são calculadas

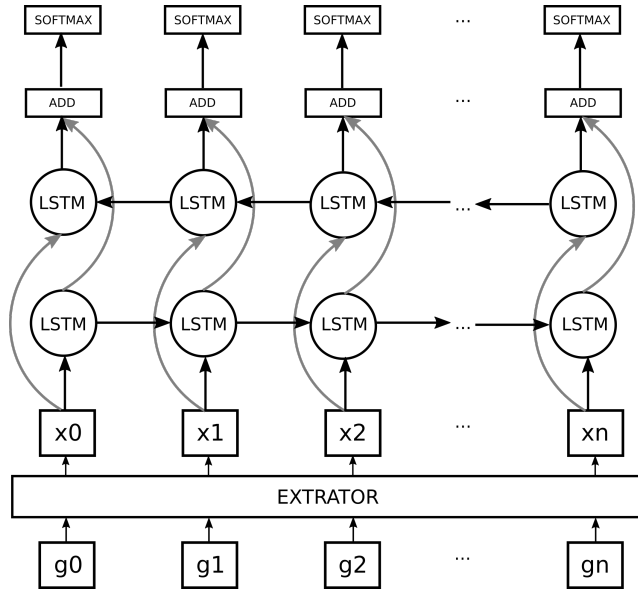


Figura 11 – Diagrama do modelo Bidirecional

utilizando a função *softmax* na saída da rede para cada t . O resultado final é uma matriz $y \in \mathbb{R}^{|\Sigma^+| \times T}$, onde T é o comprimento do gesto. Esta matriz corresponde à decodificação da rede do gesto G independente de modelo de linguagem.

Aplicamos a matriz y como as probabilidades de transição (arestas) num FST construído a partir de uma Trie do dicionário. Esse FST é utilizado pelo algoritmo *Beam-Search* para emitir as sequências com maior massa de probabilidade acumulada encontradas na busca de largura k . Em nosso trabalho, $k = 3$. Na figura 12, a decodificação fim-a-fim é apresentada como um diagrama.

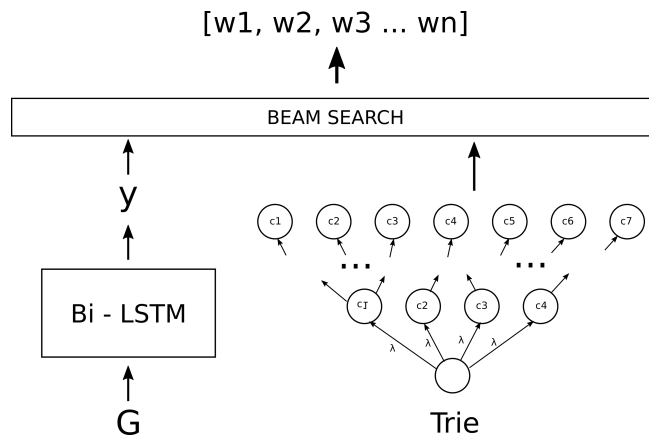


Figura 12 – Diagrama do modelo de decoding fim-a-fim

3.2 Extrator de características

Como demonstrado em QUINN; ZHAI, o processo de digitação possui um erro estrutural, modelado na Seção 3.3 por s_1 . Desta maneira, a fonte de ruído (motor, bioelétrico ou cognitivo) faz com que os valores observados quando o usuário intenta atingir uma determinada tecla sigam uma distribuição $\mathcal{N}(m(\omega_i), s_1)$, onde $m(\omega_i)$ é o ponto médio da tecla que o usuário desejava acertar.

Portanto, é apenas natural que ao alimentar o estimador gerado com a LSTM durante seu treinamento e inferência, utilizássemos não a informação discreta de qual tecla é atingida no tempo t (*one-hot vector*) em um gesto G como em ALSHARIF et al., mas sim a probabilidade de cada tecla ser a que o usuário desejava atingir no tempo t .

Assumindo que a probabilidade de cada tecla ser a tecla desejada é independente da probabilidade das demais teclas serem a tecla desejada, definimos a entrada da rede em tempo de treinamento e inferência como a matriz $X \in \mathbb{R}^{|\omega| \times T}$, onde cada elemento é

$$X_{i,t} = P(\omega_i \mid g_t), \quad (3.1)$$

onde ω_i é uma tecla no teclado virtual, ω o conjunto das teclas possíveis e g_t as coordenadas do dedo ou dispositivo apontador do usuário no tempo t como descrito na Seção 1.1. Em nosso trabalho, $|\omega|$ é 26.

QUINN; ZHAI observaram que os dados seguem uma distribuição normal centrada no ponto médio da tecla, $m(\omega_i)$. Portanto, modelamos a probabilidade de cada tecla ser a tecla desejada pelo usuário no tempo t como inversamente proporcional a distância entre g_t e $m(\omega_i)$, e podemos calculá-la como

$$P(\omega_i \mid g_t) = \frac{1}{e^{\|m(\omega_i) - g_t\| \delta}}, \quad (3.2)$$

onde δ é o coeficiente de erro estrutural.

O coeficiente de erro estrutural δ é escolhido de maneira a representar as restrições da interface e modelar da melhor maneira os erros prováveis. Ele é relacionado à s_1 , e também é condicionado à outras variáveis, como tamanho da tela e velocidade do traço. Em nosso trabalho, utilizamos $\delta = 1/4s_1 = 0.025$. Na Figura 13 vemos uma comparação entre o modelo de entrada utilizado em ALSHARIF et al., e duas diferentes parametrizações do nosso extrator de características.

Com nossa modelagem, a matriz resultante X contém a informação sobre a trajetória G em função das probabilidades da intenção do usuário em cada tempo, possibilitando a codificação de incertezas, como quando g_t se encontra na fronteira entre uma tecla e outra, atribuindo probabilidades iguais para ambas as teclas adjacentes. Esta matriz é utilizada como entrada da rede em tempo de treinamento e inferência, ao invés de G .

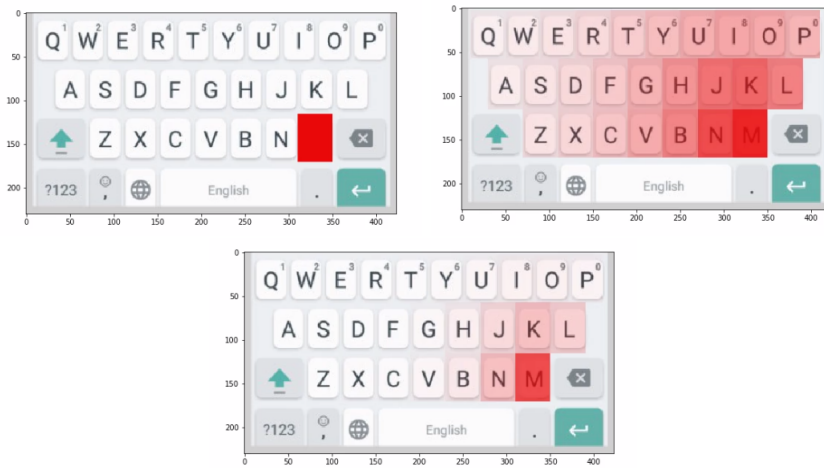


Figura 13 – Em sentido horário: Modelagem onehot (ALSHARIF et al., 2015), $\delta = 0.01$ e $\delta = 0.025$ para o mesmo ponto g_t

Além do vetor booleano (*one-hot*) ALSHARIF et al. também utiliza como entrada para a rede outros valores, como a coordenada (x, y) , tipo de gesto e tempo desde o último gesto. Essas variáveis não precisam ser utilizadas em nosso modelo.

3.3 Síntese

Para treinar nosso modelo estimador, precisamos de uma quantidade massiva de dados. Desta maneira, considerando que obter dados de usabilidade do teclado utilizado seria um processo extremamente complexo, decidimos implementar um modelo de síntese gestual capaz de gerar dados de treinamento sob demanda.

Esta seção descreve um modelo e um algoritmo de síntese de gestos humanos baseado no trabalho de QUINN; ZHAI. Esse modelo abrange apenas as variáveis patentes de uma trajetória observada G (*end-effector*), e não aspira modelar todo o processo biomecânico e bioelétrico latente que gera o movimento. Os autores obtiveram o modelo de síntese a partir de um algoritmo de segmentação que obtêm parâmetros para o processo de codificação $E(w)$ abaixo descrito.

Seja Σ o conjunto dos caracteres permitidos nas palavras de um vocabulário L , e cada palavra $w = \{c_1, c_2, c_3, \dots, c_n : c_i \in \Sigma\}$ uma sucessão ordenada de caracteres. Assumindo que w é uma palavra qualquer em L que o usuário deseja digitar – como MARCO, para cada símbolo c em w , existe uma tecla ω_c que o corresponde no teclado virtual. Tanto ã quanto Á e A têm $\omega_c = \omega_A$, por exemplo. O mapeamento ω em geral é dado pela implementação do teclado virtual.

Definimos $m : \Omega \rightarrow \mathbb{R}^2$ como uma função

$$m(\omega_c) = \left[\frac{x_2^{\omega_c} - x_1^{\omega_c}}{2} \quad \frac{y_2^{\omega_c} - y_1^{\omega_c}}{2} \right], \quad (3.3)$$

onde $(x_1^{\omega_c}, y_1^{\omega_c}), (x_2^{\omega_c}, y_2^{\omega_c})$ são as coordenadas dos pontos que correspondem as extremidades de uma tecla ω que representa o caracter c , e Ω o conjunto das teclas disponíveis no teclado virtual.

Seja K a matriz de números reais que representa as coordenadas dos centróides de cada tecla que se precisa pressionar para inserir w em um teclado convencional. Portanto, $K = [\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3, \dots, \mathbf{k}_n]^T$ e $\mathbf{k}_i = m(\omega_{c_i})$. Assume-se também, como descrito em 1.1, que a intenção do usuário é conectar os pontos indicados por K , em ordem. Esta matriz forma o protótipo do gesto, como mostra a Figura 14.



Figura 14 – Representação gráfica da matriz K para a palavra MARCO

Entretanto, como demonstrado em QUINN; ZHAI, devido ao erro motor e cognitivo do processo codificador, nem sempre o usuário atinge com precisão os centróides das teclas. Assim, os valores realmente observados nos dados de experimentos seguem uma distribuição normal, de maneira que a matriz de observações O é a realização de uma distribuição $\mathcal{N}(K, s_1)$, onde s_1 é o coeficiente de espalhamento. Na figura 15, mostramos a influência de s_1 na dispersão das amostras de O em torno de K . Em nossos experimentos de síntese, utilizamos $s_1 = 10$.



Figura 15 – Exemplo de realizações de O para $s_1 = \{5, 10, 20\}$ da esquerda para a direita, com $K_i = m(\mathbf{M})$

QUINN; ZHAI notaram em dados experimentais que as trajetórias reais raramente seguiam o protótipo em linha reta, ao contrário, apresentavam pequenas curvaturas na trajetória. Os autores observaram que a curvatura seguia uma distribuição normal, e que essa curvatura era dada por $\theta \sim \mathcal{N}(0, s_2^2)$, onde s_2 é o desvio padrão na curvatura da

trajetória, com $s_2 = 11.59^\circ$. Desta forma definimos $Z = [\mathbf{o}_1, \mathbf{v}_1, \mathbf{o}_2, \mathbf{v}_2, \mathbf{o}_3, \mathbf{v}_3, \dots, \mathbf{v}_{n-1}, \mathbf{o}_n]^T$, onde

$$\mathbf{v}_i = \frac{(\mathbf{o}_{i+1} - \mathbf{o}_i)}{2} R(\theta) + \mathbf{o}_i \quad (3.4)$$

A matriz Z representa os pontos O intercalados com os pontos da matriz V . Por sua vez, V denota o ponto médio entre quaisquer dois pontos consecutivos em O , perturbados em um ângulo θ , após a multiplicação por uma matriz de rotação $R(\theta)$. A matriz V representa os pontos conectores (*via-points*), que foram observados no trabalho de [QUINN; ZHAI](#), como as melhores âncoras para recuperar o gesto original, levando em conta a curvatura da trajetória.

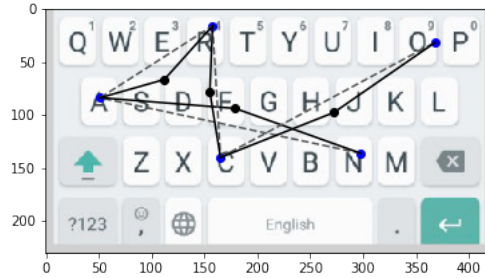


Figura 16 – Exemplo gráfico de uma matriz Z , resultado da intercalação entre O (em azul) e V , (em preto). A linha tracejada indica o protótipo da trajetória entre os pontos de O

De acordo com [FITTS](#), a trajetória do movimento humano respeita o princípio da isocronia, o qual postula que a duração de um movimento é essencialmente independente de sua amplitude. Em 1985, [FLASH; HOGAN](#) aplicam o modelo do *jerk* mínimo para aproximação de traços humanos e demonstram correlação entre os traços gerados por tal modelo e traços obtidos experimentalmente, assim como sua compatibilidade com o princípio da isocronia local e a regra dos dois terços da curvatura.

O princípio dos dois terços da curvatura ([VIVIANI; CENZATO, 1985](#)) reflete a relação entre a velocidade tangencial de um movimento e o raio da curvatura no tempo t . Isso implica que ângulos iguais são descritos em tempos iguais e a velocidade de um traço evolui de maneira inversamente proporcional à sua curvatura.

A partir dos estudos dos autores supracitados, definimos a subtrajetória G_i como o traço que conecta $\mathbf{o}_i$ a $\mathbf{o}_{i+1}$ passando por $\mathbf{v}_i$ que minimiza

$$C = \frac{1}{2} \int_{t_0}^{t_f} \left[\left(\frac{d^3x}{dt^3} \right)^2 + \left(\frac{d^3y}{dt^3} \right)^2 \right] dx, \quad (3.5)$$

a função de custo do traço G_i . Essa função representa a variação na aceleração (*jerk*) da trajetória G_i de $t = t_0$ até $t = t_f$. Por conseguinte, minimizar C maximiza a suavidade

do traço gerado.

Posteriormente, [VIVIANI; TERZUOLO](#) observam dados em seus experimentos que sugerem que gestos complexos são planejados como uma concatenação de várias primitivas mais simples. Com isso, [VIVIANI; FLASH](#) estendem a teoria do *jerk* mínimo para um número arbitrário de pontos demonstrando que a concatenação de caminhos de *jerk* mínimo prediz com sucesso as trajetórias humanas de várias formas, mantendo-se consistentes com a regra dos dois terços da curvatura e o princípio da isocronia local. Desta forma, a trajetória final G é a concatenação de todas as sub-trajetórias G_i para i de 1 até n .

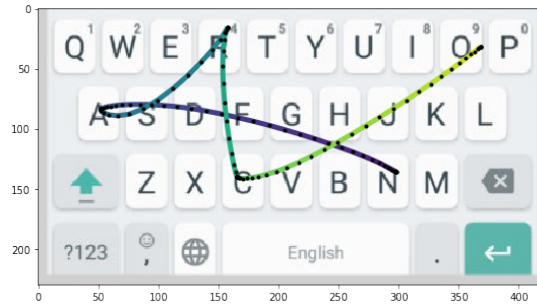


Figura 17 – Exemplo de trajetória G

Por fim, o algoritmo de síntese de gestos consiste em tomar a palavra w ; obter os centróides das teclas correspondentes a cada carácter; para cada par de centróides, em ordem, tomar seu ponto mediano e perturbá-lo em θ graus com relação à reta formada entre os dois centróides e por fim construir a trajetória de *jerk* mínimo que conecta todos os pontos, de três a três, concatenando-as.

Formalmente:

Algoritmo 1 Algoritmo de síntese de trajetórias a partir de w

```

 $w \leftarrow \text{palavra}$ 
 $K \leftarrow m(\omega_{c_i})$  para  $c_i$  em  $w$ 
 $O \leftarrow \mathcal{N}(K, s_1)$ 
 $\theta \leftarrow \mathcal{N}(0, s_2)$ 
 $V \leftarrow ((\mathbf{o}_{i+1} - \mathbf{o}_i)/2)R(\theta) + \mathbf{o}_i$  para  $i$  de 1 até  $|O| - 1$ 
 $Z \leftarrow \text{interleave}(O, V)$ 
 $G \leftarrow mjt(Z_i, Z_{i+1}, Z_{i+2})$  para  $i$  de 1 até  $|Z| - 2$ , com passo 2

```

A função $\text{interleave}(A, B)$ intercala duas matrizes A e B , e a função $\text{mjt}(p_1, p_2, p_3)$ retorna o caminho de *jerk* mínimo que conecta p_1 e p_3 passando por p_2 . Em nosso trabalho, utilizamos a implementação de trajetória de *jerk* mínimo por [YAZDANI et al.](#), que utiliza a norma infinita, ao invés norma euclideana, e resolve o problema de otimização convexa resultante.

4 Experimentos e Resultados

Neste capítulo descrevemos a metodologia experimental adotada e os experimentos realizados para avaliar os modelos propostos, inclusive se aplicados sobre dados em Português (para este fim, em particular, usando transferência de aprendizado). Em seguida, discutimos os resultados obtidos, analisando alguns erros do método e suas possíveis causas.

4.1 Metodologia Experimental

Nesta seção abordamos o vocabulário usado, os procedimentos de treinamento e avaliação dos modelos propostos.

4.1.1 Vocabulário

Utilizamos um léxico de pouco mais de 62 mil palavras para treinar e validar os algoritmos. Esse léxico foi obtido do pacote `wbritish-insane`¹, distribuído no Linux Debian e utilizado em diversos programas de código aberto. Preprocessamos as palavras de maneira a eliminar nomes próprios e palavras que utilizavam apóstrofes. Exemplos de palavras resultantes após este pré-processamento incluem: {“CONSULAR”, “FILLERS”, “BLEACH”, “UPSURGED”, “CLAIMS”, “MARCHER”, “SERVICING”, “FOLKSIEST”, “EGGPLANTS”, “MANIPULATOR”}. A Figura 18 apresenta a distribuição de comprimento das palavras pertencentes ao léxico usado.

Aqui é importante ressaltar que, em nosso trabalho de referência, [ALSHARIF et al.](#) utiliza um dataset de treinamento/avaliação diferente (*Enron*). O número de palavras únicas em seu dataset é de aproximadamente 8 mil. Neste caso, entregamos à nossa rede uma maior variabilidade de palavras, em contraste com uma menor variabilidade de gestos para cada palavra.

4.1.2 Treinamento

O treinamento do modelo foi realizado em 3 rodadas. Para cada rodada, geramos uma trajetória G sintética para cada uma das 62956 palavras do vocabulário. Separamos as primeiras 50 mil trajetórias para realizar o treinamento e validação da rede, e o restante para teste. Para cada rodada treinamos a rede durante 30 épocas. Desta maneira, em tempo de teste, a rede não treinou utilizando nenhuma das palavras do conjunto de teste, nem, por conseguinte, as trajetórias geradas a partir destas.

¹ Disponível em <https://packages.debian.org/sid/wbritish-insane>, acessado em 11/06/2018

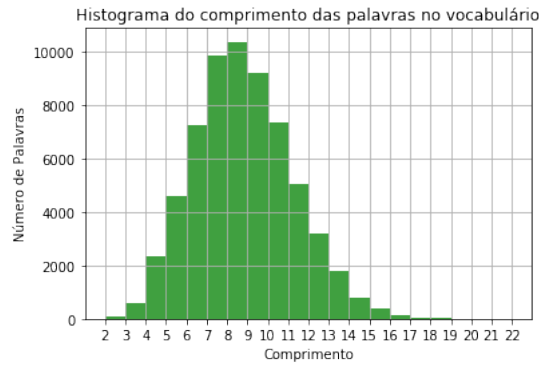


Figura 18 – Histograma do comprimento das palavras no vocabulário

Em seu trabalho, nosso *baseline* (ALSHARIF et al., 2015) treina o modelo em um dataset de 124 mil gestos, contra os nossos 150 mil. Além disso, o seu modelo é testado num conjunto de 14 mil instâncias, enquanto o nosso é avaliado num conjunto de 12 mil instâncias.

O treinamento desta rede foi realizado utilizando a pilha de código aberto Keras, rodando sobre o *backend* TensorFlow, em 8 GPUs Tesla K80 numa mesma máquina de maneira paralela. Cada rodada de treinamento, desde a síntese até a avaliação durou em média 12 horas. O tamanho do *batch* utilizado foi de 2048, dado que utilizamos paralelismo de dados e cada *batch* tem seus exemplos divididos igualmente entre as 8 GPUs.

4.1.3 Avaliação

Ao final da última das rodadas de treinamento, avaliamos o desempenho da rede treinada com e sem o uso da Trie + *Beam Search*. Consideramos um acerto da rede quando, assim como GRAVES; JAITLEY, ao eliminar caracteres repetidos e o símbolo λ , a palavra resultante após a decodificação na Trie utilizando *Beam-Search* (quando aplicável) equivale exatamente à palavra esperada. Como métrica de avaliação, usamos a acurácia, ou seja, a proporção de acertos sobre o total de casos de teste. Como casos de teste, usamos 10% dos gestos sintetizados, não utilizados para os conjuntos de treino e validação.

Também realizamos um experimento adicional considerando palavras em português. Para este caso específico, usamos como conjunto de teste as 1000 palavras mais comuns em português da web². Desta lista, removemos as palavras com acento, que não são compatíveis com a última camada da rede proposta, treinada para o idioma Inglês. Note que o modelo pode ser estendido para suportar esses símbolos, apenas treinando-se a última camada totalmente conectada da rede e utilizando-se um vocabulário de treinamento.

² Palavras obtidas em <http://hackingportuguese.com/sample-page/the-1000-most-common-nouns-in-portuguese/> acessada em 12/06/2018

4.2 Reconhecimento de Palavras

A Tabela 1 apresenta a acurácia do nosso modelo reconhecedor de palavras. Nesta tabela, *argmax* se refere a usar como resultado do classificador a saída da LSTM, sem considerar a Trie; *Trie* se refere a saída da LSTM, considerando a Trie; finalmente, *Trie-3* se refere à saída da LSTM, considerando as três palavras mais prováveis de acordo com a Trie. Como podemos ver, ao utilizar apenas a LSTM, sem restringir a saída do método às palavras existentes no dicionário utilizando a Trie, o desempenho já é significativo, com cerca de 66% de acurácia. No entanto, ao introduzirmos o modelo de linguagem proporcionado pela FST/Trie considerando as probabilidades obtidas pela LSTM, a acurácia sobe consideravelmente para cerca de 97%, um ganho superior a 50%. Ao considerar certa qualquer estimativa da Trie cuja palavra correta esteja entre as top-3 mais prováveis, o resultado sobe para cerca de 98%.

Tabela 1 – Resultados obtidos para o *dataset* de teste na rodada final

Método de <i>decoding</i>	acurácia
argmax	65.51%
Trie	96.95%
Trie-3	97.76%

4.3 Os Efeitos do Sintetizador sobre os Erros de Reconhecimento

Ao avaliarmos o sintetizador utilizado para produzir dados de treinamento, notamos que em alguns casos, a variância dos dados gerados pode causar confusão no modelo em tempo de predição. Nas Figuras de 19 até 23, podemos ver casos em que o modelo é confundido por dados do gerador.



Figura 19 – Exemplo de trajetórias para a palavra ATTIC. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso a direita, a palavra identificada foi SATIN.

Em vários casos (43.38%) em que a rede cometeu um engano em tempo de avaliação, ao testar novamente os casos de erro top-1 com novos gestos sintéticos, o modelo foi capaz atribuir a maior probabilidade à palavra correta. Isso sugere que alguns de seus erros foram provenientes de outliers das distribuições de curvatura das trajetórias (controlada por s_2) e posição de keypoints (controlada por s_1).

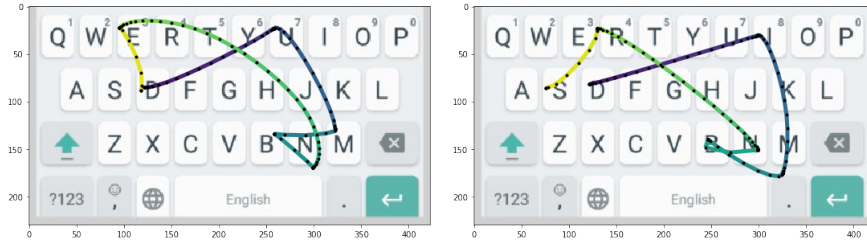


Figura 20 – Exemplo de trajetórias para a palavra **DUMBNESS**. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso à direita, a palavra identificada foi **DIMNESS**.

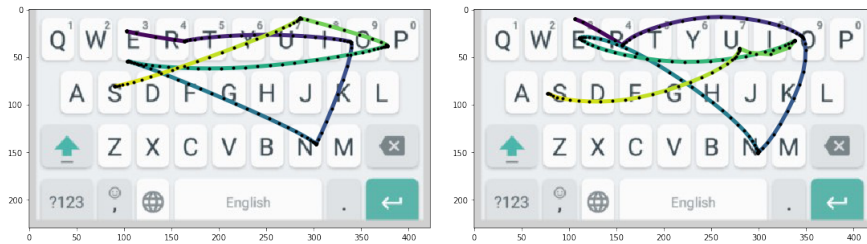


Figura 21 – Exemplo de trajetórias para a palavra **ERRONEOUS**. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso à direita, a palavra identificada foi **WRONGS**.

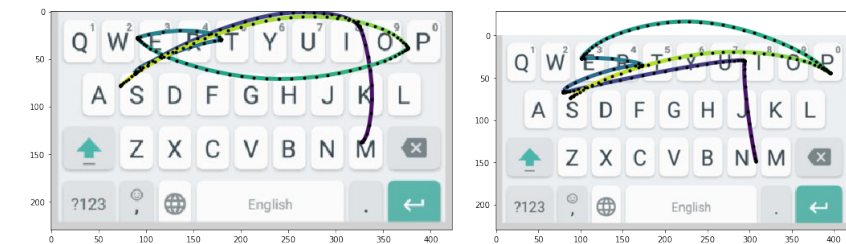


Figura 22 – Exemplo de trajetórias para a palavra **MISSTEPS**. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso à direita, a palavra identificada foi **MISTED**.

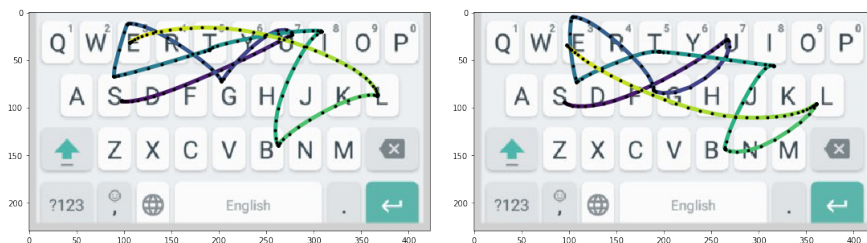


Figura 23 – Exemplo de trajetórias para a palavra **SUGGESTIBLE**. No exemplo à esquerda, o modelo decodifica corretamente. Já no caso à direita, a palavra identificada foi **SUFFERING**.

A maioria desses erros pode ser corrigida por um modelo de linguagem que considere as últimas n palavras digitadas, como uma segunda LSTM que consuma as palavras anteriores e sirva como entrada do modelo, ou uma cadeia oculta de markov. Adicio-

nar a informação de um modelo de linguagem ao método é trivial e pode ser realizado ponderando-se a busca *Beam Search*, por exemplo. Assim como [ALSHARIF et al.](#), não implementamos um modelo de linguagem mais sofisticado.

4.4 Reconhecimento de Palavras em Português com Transferência de Aprendizado

É também interessante notar que, em alguns casos de palavras fora do léxico L (*out-of-vocabulary*), ao decodificar saída da rede utilizando argmax, a LSTM produz a saída esperada. Entretanto, ao realizar a decodificação no FST construído, o método produz a palavra com maior probabilidade dentro de L . Esse é o comportamento comum e esperado em sistemas de digitação por gestos.

Isto sugere que o modelo possa ser estendido para reconhecer palavras em outros idiomas – como português, com apenas a mudança no léxico utilizado. Para provar o conceito, realizamos um teste de transferência de aprendizado (*transfer learning*) com 600 palavras comuns no português, como JOGO, VOLTA, MINISTRO e DESENVOLVIMENTO.

A transferência de aprendizado ([WEISS; KHOSHGOFTAAR; WANG, 2016](#)) envolve utilizar o conhecimento adquirido pelo modelo em um domínio – em nosso caso a conversão de gestos para palavras em inglês, para outro domínio. No experimento que realizamos, o novo domínio é a conversão de gestos para palavras em português.

Usando o conjunto das 1000 palavras mais comuns em português da web, o modelo foi capaz de atingir uma acurácia de **99.4%**. Na Figura 24, apresentamos algumas das trajetórias para tais palavras.

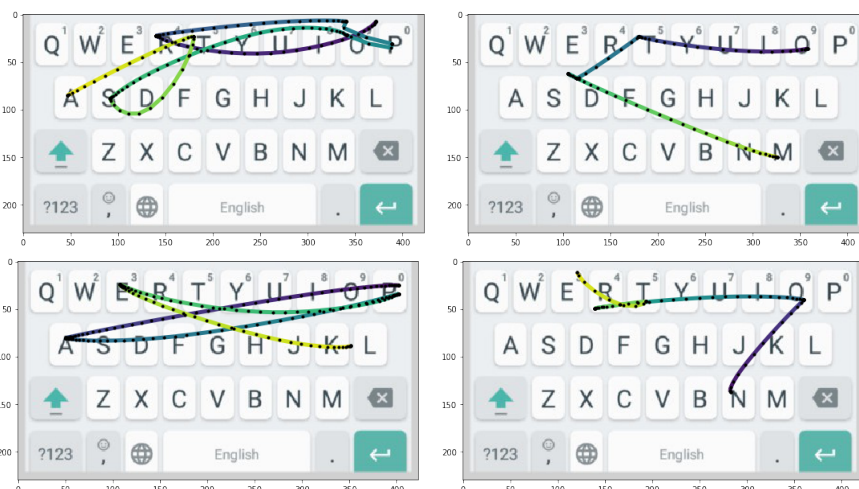


Figura 24 – Exemplo de trajetórias para a palavras em português corretamente classificadas. Em sentido horário, começando acima à esquerda: PROPOSTA, ORDEM, PAPEL e NORTE.

5 Conclusão e trabalhos futuros

O problema de decodificação de gestos é complexo, mas suas soluções agregam muito valor a interação humano-computador em dispositivos móveis. Neste trabalho, revisamos a literatura sobre o problema de decodificação de gestos em palavras, suas restrições, modelos e algoritmos propostos para sua resolução. Implementamos uma rede neural do tipo LSTM com função de custo CTC para estimar as probabilidades do mapeamento entre gestos e caracteres auxiliados pela Trie e busca Beam Search no processo decodificador.

Também implementamos um modelo gerador para produzir dados de treinamento sintéticos, capaz de produzir exemplos similares à trajetórias humanas e utilizamos estes dados para treinar nosso modelo. Pudemos com isso, realizar o treinamento em um volume razoável de exemplos e demonstramos a capacidade das redes LSTM em decodificar padrões complexos em sequências gestuais.

Realizamos uma prova de conceito de transferência de aprendizado do modelo já treinado, e pudemos constatar uma boa capacidade de generalização. Isso sugere que este modelo possa ser estendido para outros idiomas – ou problemas, sem necessidade de retreinamento completo.

Embora não possamos nos comparar diretamente com o estudo de [ALSHARIF et al.](#), devido a diferenças de metodologia, menores diferenças arquiteturais, acesso aos dados e ao seu modelo gerador, acreditamos que a modelagem probabilística para o extrator de características apresentado na Seção 3.2, possa ser a razão de uma melhoria de **3,45%** quando comparado ao seu modelo em um dataset sintético (*Enron*). Isto, é claro, deve ser objeto de investigação mais profunda.

Entre diversos outros modelos capazes de resolver o problema de digitação por gestos, talvez os mais promissores sejam aqueles baseados em *Neural Machine Translation*. Um deles em específico, a arquitetura *Seq2Seq* ([SUTSKEVER; VINYALS; LE, 2014](#)), pode ser ainda mais assertiva para servir de estimador para as probabilidades.

Finalmente, o modelo apresentado pode ser adaptado para problemas de natureza similar, como reconhecimento de palavras em digitação com os olhos. Com mínimas alterações, também se pode utilizar nosso método de síntese e reconhecimento para reconhecer gestos genéricos em dispositivos móveis.

Referências

- ALSHARIF, O. et al. Long short term memory neural network for keyboard gesture decoding. In: IEEE. *Acoustics, Speech and Signal Processing (ICASSP), 2015 IEEE International Conference on*. [S.l.], 2015. p. 2076–2080. Citado 12 vezes nas páginas 13, 22, 23, 24, 32, 35, 37, 38, 43, 44, 47 e 49.
- CHATTY, S.; LECOANET, P. Pen computing for air traffic control. In: ACM. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. [S.l.], 1996. p. 87–94. Citado na página 25.
- FITTS, P. M. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of experimental psychology*, American Psychological Association, v. 47, n. 6, p. 381, 1954. Citado na página 40.
- FLASH, T.; HOGAN, N. The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of neuroscience*, Soc Neuroscience, v. 5, n. 7, p. 1688–1703, 1985. Citado na página 40.
- GOLDBERG, D.; RICHARDSON, C. Touch-typing with a stylus. In: ACM. *Proceedings of the INTERACT'93 and CHI'93 conference on Human factors in computing systems*. [S.l.], 1993. p. 80–87. Citado na página 25.
- GOODFELLOW, I. et al. *Deep learning*. [S.l.]: MIT press Cambridge, 2016. v. 1. Citado na página 24.
- GRAVES, A. et al. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In: ACM. *Proceedings of the 23rd international conference on Machine learning*. [S.l.], 2006. p. 369–376. Citado 3 vezes nas páginas 24, 32 e 33.
- GRAVES, A.; JAITLEY, N. Towards end-to-end speech recognition with recurrent neural networks. In: *International Conference on Machine Learning*. [S.l.: s.n.], 2014. p. 1764–1772. Citado 4 vezes nas páginas 23, 24, 34 e 44.
- HAWKINS, J. C.; SIPHER, J. K.; RON, M. I. *Multiple pen stroke character set and handwriting recognition system with immediate response*. [S.l.]: Google Patents, 2002. US Patent 6,493,464. Citado na página 25.
- HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural computation*, MIT Press, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 24.
- HORNIK, K. Approximation capabilities of multilayer feedforward networks. *Neural networks*, Elsevier, v. 4, n. 2, p. 251–257, 1991. Citado na página 24.
- KRISTENSSON, P.-O.; ZHAI, S. Shark 2: a large vocabulary shorthand writing system for pen-based computers. In: ACM. *Proceedings of the 17th annual ACM symposium on User interface software and technology*. [S.l.], 2004. p. 43–52. Citado na página 29.

- KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: *Advances in neural information processing systems*. [S.l.: s.n.], 2012. p. 1097–1105. Citado na página 34.
- MANKOFF, J.; ABOWD, G. D. Cirrin: a word-level unistroke keyboard for pen input. In: ACM. *Proceedings of the 11th annual ACM symposium on User interface software and technology*. [S.l.], 1998. p. 213–214. Citado na página 27.
- MITRA, S.; ACHARYA, T. Gesture recognition: A survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, IEEE, v. 37, n. 3, p. 311–324, 2007. Citado na página 29.
- MONTGOMERY, E. B. Bringing manual input into the 20th century: New keyboard concepts. *Computer*, ERIC, v. 15, n. 3, p. 11–18, 1982. Citado na página 28.
- PASCANU, R.; MIKOLOV, T.; BENGIO, Y. Understanding the exploding gradient problem. *CoRR*, abs/1211.5063, 2012. Citado 2 vezes nas páginas 24 e 31.
- PEDERSEN, E. R. et al. Tivoli: An electronic whiteboard for informal workgroup meetings. In: *Readings in Human–Computer Interaction*. [S.l.]: Elsevier, 1995. p. 509–516. Citado na página 25.
- PERLIN, K. Quikwriting: continuous stylus-based text entry. In: ACM. *Proceedings of the 11th annual ACM symposium on User interface software and technology*. [S.l.], 1998. p. 215–216. Citado na página 27.
- QUINN, P.; ZHAI, S. Modeling gesture-typing movements. *Human–Computer Interaction*, Taylor & Francis, v. 33, n. 3, p. 234–280, 2018. Citado 7 vezes nas páginas 23, 24, 34, 37, 38, 39 e 40.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *nature*, Nature Publishing Group, v. 323, n. 6088, p. 533, 1986. Citado na página 30.
- SEARS, A.; ARORA, R. Data entry for mobile devices: an empirical comparison of novice performance with jot and graffiti. *Interacting with Computers*, v. 14, n. 5, p. 413–433, 2002. Disponível em: <[http://dx.doi.org/10.1016/S0953-5438\(01\)00060-1](http://dx.doi.org/10.1016/S0953-5438(01)00060-1)>. Citado na página 25.
- SHORE, J.; JOHNSON, R. Axiomatic derivation of the principle of maximum entropy and the principle of minimum cross-entropy. *IEEE Transactions on information theory*, IEEE, v. 26, n. 1, p. 26–37, 1980. Citado na página 32.
- SUTSKEVER, I. et al. On the importance of initialization and momentum in deep learning. In: *International Conference on Machine Learning*. [S.l.: s.n.], 2013. p. 1139–1147. Citado na página 26.
- SUTSKEVER, I.; VINYALS, O.; LE, Q. V. Sequence to sequence learning with neural networks. In: *Advances in neural information processing systems*. [S.l.: s.n.], 2014. p. 3104–3112. Citado na página 49.
- TAPPERT, C. C.; SUEN, C. Y.; WAKAHARA, T. The state of the art in online handwriting recognition. *IEEE Transactions on pattern analysis and machine intelligence*, IEEE, v. 12, n. 8, p. 787–808, 1990. Citado na página 30.

- VIVIANI, P.; CENZATO, M. Segmentation and coupling in complex movements. *Journal of experimental psychology: Human perception and performance*, American Psychological Association, v. 11, n. 6, p. 828, 1985. Citado na página 40.
- VIVIANI, P.; FLASH, T. Minimum-jerk, two-thirds power law, and isochrony: converging approaches to movement planning. *Journal of Experimental Psychology: Human Perception and Performance*, American Psychological Association, v. 21, n. 1, p. 32, 1995. Citado na página 41.
- VIVIANI, P.; TERZUOLO, C. A. On the relation between word-specific patterns and the central control model of typing: A reply to gentner. American Psychological Association, 1982. Citado na página 41.
- WANG, T. et al. End-to-end text recognition with convolutional neural networks. In: IEEE. *Pattern Recognition (ICPR), 2012 21st International Conference on*. [S.l.], 2012. p. 3304–3308. Citado na página 23.
- WEISS, K.; KHOSHGOFTAAR, T. M.; WANG, D. A survey of transfer learning. *Journal of Big Data*, Nature Publishing Group, v. 3, n. 1, p. 9, 2016. Citado na página 47.
- WOLF, C. G.; RHYNE, J. R. Gesturing with shared drawing tools. In: ACM. *INTERACT'93 and CHI'93 Conference Companion on Human Factors in Computing Systems*. [S.l.], 1993. p. 137–138. Citado na página 25.
- YAZDANI, M. et al. A simple control policy for achieving minimum jerk trajectories. *Neural Networks*, Elsevier, v. 27, p. 74–80, 2012. Citado na página 41.
- ZHAI, S.; HUNTER, M.; SMITH, B. A. Performance optimization of virtual keyboards. *Human–Computer Interaction*, Taylor & Francis, v. 17, n. 2-3, p. 229–269, 2002. Citado na página 28.
- ZHAI, S.; KRISTENSSON, P.-O. Shorthand writing on stylus keyboard. In: ACM. *Proceedings of the SIGCHI conference on Human factors in computing systems*. [S.l.], 2003. p. 97–104. Citado 2 vezes nas páginas 27 e 29.
- ZHAI, S.; KRISTENSSON, P. O. The word-gesture keyboard: reimagining keyboard interaction. *Communications of the ACM*, ACM, v. 55, n. 9, p. 91–101, 2012. Citado na página 28.
- ZHAI, S. et al. Foundational issues in touch-surface stroke gesture design—an integrative review. *Foundations and Trends® in Human–Computer Interaction*, Now Publishers, Inc., v. 5, n. 2, p. 97–205, 2012. Citado na página 26.